

Course Outline: SQL Fundamentals

Title: SQL Fundamentals **Skill:** 30 — Consultar tablas y gestionar filas en bases de datos **Learnpack:** + Consultando bases de datos con SQL en PostgreSQL (Fundamentos de SQL) **File:** s30-w11d31-consultando-bases-de-datos-con-sql-en-po **Prerequisite:** Bases de datos (s30-w11d31-bases-de-datos) **Target Audience:** Beginner developers who understand what relational databases are and are ready to write their first SQL queries **Total Lessons:** 14 **Format:** Mixed (21% hands-on = 3 code challenges)

Course Description

You already know what databases are and how relational databases differ from document stores and key-value systems. Now it's time to speak their language. SQL (Structured Query Language) is what you use to interact with relational databases like PostgreSQL — creating rows, reading records, updating values, deleting entries, filtering results, and summarizing data. This course takes you from zero SQL to writing real queries with confidence.

Course Philosophy

This course emphasizes:

- Reading data before writing it: understanding `SELECT` deeply before touching `INSERT` or `DELETE`
 - Precision over convenience: learning why `SELECT *` is a habit to avoid from day one
 - `NULL` as a first-class concept: treating `NULL` not as a bug but as a fundamental property of relational data
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - The Relational Model	2
02 - Reading and Writing Data	3
03 - Filtering and Sorting	3
04 - Aggregation and <code>NULL</code>	3
05 - Assessment	1
06 - Conclusion	1
Total	14

Section 00: Welcome

00.0 Welcome to SQL Fundamentals

Learning Objectives:

- Understand what this course covers and why it matters
- Preview the journey from the relational model to full data querying

Content Outline:

- The gap: you know what a relational database is — now let's talk to one
- What SQL is and where it fits in the development stack
- What students will be able to do by the end of this course
- Overview of sections: relational model → CRUD → filtering/sorting → aggregation → assessment

Transitions: Leads directly into understanding the structure behind every SQL table.

Key Principles:

- SQL is the standard language for relational databases — PostgreSQL, MySQL, and SQL Server all speak it
- Knowing SQL is a foundational skill for any backend or full-stack developer

Examples:

- A backend receives a request for all users who signed up last month → it runs a SQL query to fetch them
 - An admin dashboard shows total revenue by product → that summary is produced by a SQL aggregation
-

Section 01: The Relational Model

01.0 SQL and the Relational Model

Learning Objectives:

- Explain what SQL is and what it is used for
- Describe the structure of a relational table: tables, rows, and columns
- Explain the role of a primary key and auto-incrementing IDs

Content Outline:

- What SQL is: a declarative language for querying and manipulating relational databases
- SQL vs. the database engine: SQL is the language, PostgreSQL is the engine that runs it
- Tables: the fundamental unit of storage — structured and typed, not freeform
- Rows: individual records (a single user, a single order)
- Columns: attributes of each record (name, email, created_at)
- Primary keys: every row needs a unique identifier
 - Must be unique across all rows in the table

- Must never be NULL
- Immutable — a PK should not change after a row is created
- Auto-incrementing IDs: letting the database assign the PK automatically
 - **SERIAL** in PostgreSQL generates the next integer automatically
 - **BIGSERIAL** for high-volume tables (supports very large numbers)
 - PostgreSQL sequences: how the engine manages the next value internally
- Declarative vs. imperative: with SQL you describe *what* you want, not *how* to get it

Transitions: Now that we understand how tables are structured, the next question is: what kinds of data can each column actually hold?

Key Principles:

- Every table has a primary key — it is the canonical and unambiguous way to reference a specific row
- Auto-increment removes the burden of managing IDs from application code
- SQL is declarative: the database engine figures out the execution plan

Examples:

- A `users` table: `id SERIAL PRIMARY KEY, name TEXT, email TEXT, created_at TIMESTAMPTZ`
 - Fetching the row with `id = 42` is always unambiguous because PKs are unique and non-null
-

01.1 Data Types in SQL

Learning Objectives:

- Identify the four main categories of SQL data types: numeric, text, boolean, date/time
- Choose the appropriate data type for a given column
- Explain why choosing the right type matters for correctness and performance

Content Outline:

- Why types matter: the database enforces them — wrong type = error or silent data corruption
- Numeric types:
 - **INTEGER** / **INT** — whole numbers (user IDs, counts, ages)
 - **BIGINT** — large integers for high-volume IDs or very large counts
 - **NUMERIC(precision, scale)** / **DECIMAL** — exact decimals (prices, financial data)
 - **FLOAT** / **REAL** — approximate decimals (scientific measurements, coordinates)
 - Warning: never use **FLOAT** for money — floating-point arithmetic is approximate
- Text types:
 - **TEXT** — unlimited-length string; PostgreSQL's recommended default for strings
 - **VARCHAR(n)** — string with a maximum character limit (use when the limit has real meaning)
 - **CHAR(n)** — fixed-length, padded with spaces; mostly legacy, rarely used today
- Boolean:
 - **BOOLEAN** — stores true, false, or NULL

- PostgreSQL accepts `TRUE`, `FALSE`, `'yes'`, `'no'`, `1`, `0` as input
- Date and time:
 - `DATE` — calendar date only (2024-01-15)
 - `TIME` — time of day only, no date
 - `TIMESTAMP` — date + time without timezone information
 - `TIMESTAMPTZ` — date + time with timezone (recommended default for most applications)

Transitions: With the relational model and data types understood, we are ready to write actual SQL — starting with the most important operation: reading data.

Key Principles:

- Use `TEXT` over `VARCHAR` in PostgreSQL unless a hard length limit has a real business meaning
- Never use `FLOAT` for financial data — use `NUMERIC` for exact arithmetic
- Default to `TIMESTAMPTZ` for any timestamp column — timezone awareness prevents silent bugs

Examples:

- `price NUMERIC(10, 2)` stores 99.99 exactly; `FLOAT` might store 99.98999...
- `created_at TIMESTAMPTZ DEFAULT NOW()` captures the exact moment a record was inserted, timezone-aware
- `is_active BOOLEAN DEFAULT TRUE` — clean, explicit, and enforced by the database

Section 02: Reading and Writing Data

02.0 Selecting Data

Learning Objectives:

- Write a basic `SELECT` statement to retrieve rows from a table
- Explain why selecting specific columns is better than using `SELECT *`
- Use column aliases to rename output columns

Content Outline:

- Basic `SELECT` syntax: `SELECT column1, column2 FROM table_name;`
- Reading all rows: no `WHERE` clause returns every row in the table
- Selecting all columns with `*`: quick to type, but problematic
 - Returns columns you do not need — wastes bandwidth and memory
 - Schema changes silently break code that relied on column order
 - Makes it harder to understand what data is actually being used
 - Rule: always name the columns you need explicitly
- Selecting specific columns: `SELECT id, name, email FROM users;`
- Column aliases: renaming a column in the output
 - `SELECT name AS full_name, email AS contact FROM users;`
 - Aliases do not change the actual column name in the table
- Previewing data: using `LIMIT` to sample rows (covered fully in the next section)
 - `SELECT id, name FROM users LIMIT 5;` — safe way to explore a large table

Transitions: Reading data is only half of CRUD. The next lesson covers INSERT, UPDATE, and DELETE — and introduces a safer alternative to permanent deletion.

Key Principles:

- SELECT is the most frequently used SQL statement — build good habits with it from the start
- `SELECT *` is acceptable in a psql shell for quick exploration; it is not acceptable in production code
- SQL keywords are case-insensitive, but the convention is UPPERCASE for keywords and lowercase for table/column names

Examples:

- `SELECT id, email, created_at FROM users;` — explicit, intentional, maintainable
 - `SELECT first_name || ' ' || last_name AS full_name FROM employees;` — string concatenation with alias
 - `SELECT * FROM users LIMIT 10;` — acceptable for exploration; not for application queries
-

02.1 Modifying Data

Learning Objectives:

- Write INSERT, UPDATE, and DELETE statements correctly
- Explain the concept of soft-delete and when to use it instead of DELETE
- Recognize the danger of UPDATE and DELETE without a WHERE clause

Content Outline:

- INSERT: adding new rows
 - `INSERT INTO users (name, email) VALUES ('Ana', 'ana@example.com');`
 - Inserting multiple rows in one statement: `VALUES ('Ana', ...), ('Luis', ...)`
 - The RETURNING clause: get back the generated ID without a second query
 - `INSERT INTO users (name, email) VALUES ('Ana', 'ana@example.com') RETURNING id;`
- UPDATE: modifying existing rows
 - `UPDATE users SET email = 'new@example.com' WHERE id = 5;`
 - Always use WHERE with UPDATE — without it, every single row in the table gets updated
 - Updating multiple columns: `UPDATE users SET name = 'Ana G.', updated_at = NOW() WHERE id = 5;`
- DELETE: removing rows permanently
 - `DELETE FROM users WHERE id = 5;`
 - Always use WHERE with DELETE — without it, every single row in the table is deleted
 - `DELETE FROM users;` — wipes the entire table with no warning and no undo
- Soft-delete: marking a row as deleted without removing it
 - What it is: a column that records when (or whether) a row was deleted
 - Common implementations:
 - `deleted_at TIMESTAMPTZ` — NULL means not deleted; a timestamp means deleted at that moment

- `is_deleted BOOLEAN DEFAULT FALSE` — simpler but loses the deletion timestamp
- When to use soft-delete:
 - You need audit history or the ability to recover deleted data
 - Other tables reference this row (physical deletion would break data integrity)
 - Legal or compliance requirements mandate data retention
- When physical DELETE is appropriate:
 - Temporary or session data with no dependencies (expired tokens, temp files)
 - Log entries past a defined retention window
- Executing a soft-delete: `UPDATE users SET deleted_at = NOW() WHERE id = 5;`
- Querying non-deleted rows: `SELECT id, name FROM users WHERE deleted_at IS NULL;`

Transitions: You can now read, insert, update, and delete data. The next step is reading with precision — filtering rows to return exactly what you need.

Key Principles:

- An UPDATE or DELETE without WHERE is one of the most dangerous SQL mistakes — always double-check before running
- Soft-delete preserves history and enables recovery; hard DELETE is permanent and irreversible
- The RETURNING clause eliminates the need for a follow-up SELECT after an INSERT

Examples:

- A user deletes their account → `UPDATE users SET deleted_at = NOW() WHERE id = 42` — the row still exists for compliance auditing
- An expired session token → `DELETE FROM sessions WHERE expires_at < NOW()` — physical delete is appropriate here
- `INSERT INTO orders (user_id, total) VALUES (42, 199.99) RETURNING id, created_at;`

02.2 CRUD Challenge

Challenge Prompt: Given the `products` table (already created and seeded with data), write the four queries that INSERT a new product, SELECT its name and price, UPDATE its price, and DELETE it.

Solution Code:

```
INSERT INTO products (name, price, in_stock)
VALUES ('Wireless Keyboard', 49.99, TRUE)
RETURNING id;

SELECT name, price FROM products WHERE id = 1;

UPDATE products SET price = 39.99 WHERE id = 1;

DELETE FROM products WHERE id = 1;
```

Challenge Code:

```
-- INSERT a new product named 'Wireless Keyboard', price 49.99, in_stock =  
true  
-- your code here  
  
-- SELECT only name and price for the product you just inserted (use its id)  
-- your code here  
  
-- UPDATE its price to 39.99  
-- your code here  
  
-- DELETE it  
-- your code here
```

Section 03: Filtering and Sorting

03.0 Filtering with WHERE

Learning Objectives:

- Use the WHERE clause to filter rows by one or more conditions
- Apply mathematical, logical, range, and text operators in queries
- Combine multiple conditions correctly using AND, OR, and NOT

Content Outline:

- The WHERE clause: `SELECT ... FROM ... WHERE condition;`
- Only rows where the condition is TRUE are returned — UNKNOWN and FALSE rows are excluded
- Mathematical (comparison) operators:
 - `=` — exact match: `WHERE status = 'active'`
 - `!=` or `<>` — not equal: `WHERE status != 'deleted'`
 - `<`, `>` — strictly less/greater than: `WHERE age > 18`
 - `<=`, `>=` — less/greater than or equal: `WHERE price <= 100`
- Logical operators — combining conditions:
 - `AND` — both conditions must be TRUE: `WHERE price > 10 AND in_stock = TRUE`
 - `OR` — at least one condition must be TRUE: `WHERE category = 'books' OR category = 'ebooks'`
 - `NOT` — negates a condition: `WHERE NOT is_deleted`
 - Operator precedence: AND is evaluated before OR — use parentheses to make intent explicit
 - `WHERE a OR b AND c` is evaluated as `WHERE a OR (b AND c)` — potentially surprising
 - Always write: `WHERE (a OR b) AND c` when that is what you mean
- Range operators:
 - `IN` — matches any value in a list: `WHERE status IN ('active', 'pending')`

- **NOT IN** — excludes all values in a list: `WHERE status NOT IN ('deleted', 'banned')`
- **BETWEEN** — inclusive range: `WHERE price BETWEEN 10 AND 50` (equivalent to `>= 10 AND <= 50`)
- **NOT BETWEEN** — outside the range
- Text operators:
 - **LIKE** — case-sensitive pattern matching: `WHERE name LIKE 'Ana%'`
 - **ILIKE** — case-insensitive pattern matching (PostgreSQL-specific): `WHERE name ILIKE '%garcia%'`
 - Wildcards: `%` matches any sequence of characters; `_` matches exactly one character
 - `LIKE 'A%'` — starts with A; `LIKE '%son'` — ends with son; `LIKE '%ar%'` — contains ar
 - Note: **ILIKE** is a PostgreSQL extension; standard SQL uses `LOWER(col) LIKE LOWER(pattern)`

Transitions: Filtering gives you the right rows. The next step is controlling the order in which they are returned and how many you receive.

Key Principles:

- Always use parentheses when mixing AND and OR — ambiguity causes subtle, hard-to-find bugs
- Prefer **ILIKE** over **LIKE** for user-facing text searches where case should not matter
- **IN (list)** is cleaner and more readable than chaining many OR conditions on the same column

Examples:

- `WHERE created_at >= '2024-01-01' AND created_at < '2025-01-01'` — all records from 2024
- `WHERE last_name ILIKE '%mart%'` — matches 'Martin', 'Smart', 'martinez', 'SMART'
- `WHERE status IN ('shipped', 'delivered')` — cleaner than `WHERE status = 'shipped' OR status = 'delivered'`
- `WHERE (role = 'admin' OR role = 'moderator') AND is_active = TRUE` — explicit grouping prevents logic errors

03.1 Ordering and Limiting Results

Learning Objectives:

- Sort query results using **ORDER BY** in ascending and descending order
- Limit the number of rows returned using **LIMIT**
- Combine **ORDER BY** and **LIMIT** for top-N queries and basic pagination

Content Outline:

- **ORDER BY:** sorting the result set
 - `SELECT ... FROM ... ORDER BY column ASC;` — ascending order (default when not specified)
 - `SELECT ... FROM ... ORDER BY column DESC;` — descending order
 - Sorting by multiple columns: `ORDER BY last_name ASC, first_name ASC` — resolves ties predictably

- NULL values in ORDER BY: PostgreSQL places NULLs last in ASC and first in DESC by default
 - Override with: `ORDER BY score DESC NULLS LAST`
- LIMIT: restricting the number of rows returned
 - `SELECT ... FROM ... LIMIT 10;` — return at most 10 rows
 - Without ORDER BY, LIMIT returns an arbitrary set of rows — never rely on this for consistent results
 - Always pair LIMIT with ORDER BY when the specific rows matter
- OFFSET: skipping rows
 - `LIMIT 10 OFFSET 20` — skip the first 20 rows, return the next 10
 - Practical use: page 3 of results with 10 items per page → `LIMIT 10 OFFSET 20`
 - Note: OFFSET-based pagination slows down at very large offsets — adequate for now
- Common patterns:
 - Top 5 most expensive products: `ORDER BY price DESC LIMIT 5`
 - 10 most recent sign-ups: `ORDER BY created_at DESC LIMIT 10`
 - Alphabetical first page: `ORDER BY name ASC LIMIT 20 OFFSET 0`

Transitions: You can now read exactly the rows you want, in the order you want them. Next, we explore aggregation — computing counts, totals, and averages across groups of rows.

Key Principles:

- LIMIT without ORDER BY is non-deterministic — the database can return any rows in any order
- Multiple ORDER BY columns resolve ties and produce consistent, reproducible results
- OFFSET starts at 0: `OFFSET 0` returns from the beginning, `OFFSET 10` skips the first 10 rows

Examples:

- `SELECT name, salary FROM employees ORDER BY salary DESC LIMIT 3;` — the three highest earners
- `SELECT * FROM orders ORDER BY created_at DESC LIMIT 10 OFFSET 0;` — first page of recent orders
- `SELECT id, title FROM articles ORDER BY published_at DESC NULLS LAST LIMIT 20;` — published articles first, drafts at the end

03.2 Filtering Challenge

Challenge Prompt: Using the `orders` table (already seeded), write a single query that returns the 5 most recent orders with status `'pending'` or `'processing'`, showing only `id`, `customer_name`, `total`, and `created_at`.

Solution Code:

```
SELECT id, customer_name, total, created_at
FROM orders
WHERE status IN ('pending', 'processing')
```

```
ORDER BY created_at DESC
LIMIT 5;
```

Challenge Code:

```
-- Return the 5 most recent orders where status is 'pending' or 'processing'
-- Show only: id, customer_name, total, created_at
-- your code here
```

Section 04: Aggregation and NULL

04.0 Grouping and Aggregating

Learning Objectives:

- Use aggregate functions to compute summaries across rows
- Group rows by a column value using GROUP BY
- Filter aggregated groups using HAVING
- Use DISTINCT to deduplicate values

Content Outline:

- What aggregation means: computing a single result from multiple rows
- Aggregate functions:
 - `COUNT(*)` — total number of rows in the result, including NULLs
 - `COUNT(column)` — number of non-NULL values in a specific column
 - `SUM(column)` — sum of all non-NULL numeric values
 - `AVG(column)` — arithmetic mean of non-NULL numeric values
 - `MIN(column)` — smallest non-NULL value (works on numbers, text, dates)
 - `MAX(column)` — largest non-NULL value
 - `DISTINCT` — deduplicate: `SELECT DISTINCT status FROM orders;`
 - `COUNT(DISTINCT column)` — count of unique non-NULL values: `COUNT(DISTINCT user_id)`
- GROUP BY: computing aggregates per group
 - `SELECT category, COUNT(*) FROM products GROUP BY category;`
 - Every non-aggregated column in SELECT must appear in GROUP BY — this is a hard SQL rule
 - Common mistake: `SELECT category, name, COUNT(*) FROM products GROUP BY category;` — error, `name` is not in GROUP BY
- HAVING: filtering groups after aggregation
 - WHERE filters rows before grouping; HAVING filters groups after grouping
 - `SELECT category, COUNT(*) FROM products GROUP BY category HAVING COUNT(*) > 5;`
 - You cannot use aggregate functions inside a WHERE clause — use HAVING instead

- You can use both WHERE and HAVING in the same query:
 - WHERE removes rows before grouping
 - HAVING removes groups after aggregation
- Conceptual execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

Transitions: Aggregation is powerful, but there is one concept that silently affects every aggregate function — NULL. Understanding NULL is the final piece before the assessment.

Key Principles:

- Aggregate functions (except COUNT(*)) silently ignore NULL values — always verify this is the intended behavior
- WHERE and HAVING are not interchangeable: WHERE acts on rows before grouping, HAVING on groups after
- Every non-aggregated column in SELECT must appear in GROUP BY — the database will reject it otherwise

Examples:

- SELECT status, COUNT(*) FROM orders GROUP BY status; — number of orders per status
- SELECT user_id, SUM(total) AS lifetime_value FROM orders GROUP BY user_id HAVING SUM(total) > 500; — high-value customers
- SELECT category, AVG(price) FROM products WHERE in_stock = TRUE GROUP BY category ORDER BY AVG(price) DESC;

04.1 NULL in SQL

Learning Objectives:

- Explain what NULL means in a relational database
- Use IS NULL and IS NOT NULL to correctly filter rows with missing values
- Describe how NULL behaves in comparison operators and aggregate functions

Content Outline:

- What NULL means: the absence of a value — not zero, not an empty string, not false
 - NULL represents unknown or not applicable
 - Examples: middle_name (not everyone has one), deleted_at (not yet deleted), phone (not provided)
- Why = NULL never works — the most common NULL mistake:
 - NULL is not equal to anything, not even itself
 - WHERE email = NULL always returns zero rows, even when NULL values exist
 - Reason: NULL comparisons evaluate to UNKNOWN, not TRUE or FALSE
 - UNKNOWN rows are excluded by WHERE — same as FALSE
 - The fix: always use IS NULL or IS NOT NULL to check for NULL
- IS NULL and IS NOT NULL:
 - WHERE deleted_at IS NULL — rows that have not been soft-deleted

- `WHERE phone IS NOT NULL` — rows where a phone number was provided
- These are the only correct operators for NULL checking
- NULL in logical expressions:
 - `NULL AND TRUE` → UNKNOWN
 - `NULL AND FALSE` → FALSE (short-circuit: anything AND false is false)
 - `NULL OR TRUE` → TRUE (short-circuit: anything OR true is true)
 - `NULL OR FALSE` → UNKNOWN
 - `NOT NULL` → UNKNOWN
 - Any arithmetic with NULL (`NULL + 5, NULL * 2`) → NULL
- NULL in aggregate functions:
 - `COUNT(*)` — counts all rows, including rows with NULL values in any column
 - `COUNT(column)` — counts only rows where that column is NOT NULL
 - `SUM, AVG, MIN, MAX` — all silently skip NULL values
 - Consequence: `AVG(score)` computes the average only of students who have a score, not all students
 - This can produce misleading results — always be explicit about what you are averaging
- COALESCE: substituting a default value for NULL
 - `COALESCE(value, fallback)` — returns value if not NULL, otherwise fallback
 - `SELECT COALESCE(phone, 'N/A') FROM contacts;` — replaces NULL phone with 'N/A'
 - `SELECT COALESCE(score, 0) FROM students;` — treats missing scores as 0

Transitions: You now have the complete picture of single-table SQL in PostgreSQL. The assessment tests everything from the relational model through NULL handling.

Key Principles:

- NULL means "unknown" — it cannot be equal to, greater than, or less than anything, including itself
- Always use `IS NULL` / `IS NOT NULL` — never `= NULL` or `!= NULL`
- Aggregate functions silently skip NULLs — confirm this matches your intent before trusting the result

Examples:

- `WHERE last_login IS NULL` — users who have never logged in
- `SELECT AVG(score) FROM students` — returns the average of students with a score (NULLs excluded silently)
- `SELECT COALESCE(discount, 0) FROM orders;` — treats orders with no discount as 0% off
- `SELECT COUNT(*), COUNT(phone) FROM users;` — difference reveals how many users have no phone on file

04.2 Aggregation Challenge

Challenge Prompt: Using the `sales` table (columns: `id`, `product_name`, `category`, `amount`, `sold_at`), write a query that returns each category with its total sales amount and number of sales, for categories with more than 3 sales, ordered by total amount descending.

Solution Code:

```
SELECT
    category,
    COUNT(*) AS num_sales,
    SUM(amount) AS total_amount
FROM sales
GROUP BY category
HAVING COUNT(*) > 3
ORDER BY total_amount DESC;
```

Challenge Code:

```
-- Return each category with its total sales amount and number of sales
-- Only include categories that have more than 3 sales
-- Order results by total_amount descending
-- your code here
```

Section 05: Assessment

05.0 SQL Fundamentals Assessment 🧠

Learning Objectives:

- Demonstrate understanding of relational tables, primary keys, and data types
- Write and interpret SELECT, INSERT, UPDATE, and DELETE statements
- Apply WHERE filtering, ORDER BY, LIMIT, GROUP BY, HAVING, and NULL handling

Question Format: Multiple choice

Questions:

1. Which SQL statement is used to retrieve data from a table?
 - a) FETCH
 - b) GET
 - c) SELECT ✓
 - d) READ
2. What is the purpose of a PRIMARY KEY?
 - a) To store the largest value in the column
 - b) To uniquely identify each row in the table ✓
 - c) To link two tables together
 - d) To set a default value for a column
3. Which data type should you use to store exact monetary values in PostgreSQL?

- a) FLOAT
- b) REAL
- c) NUMERIC ✓
- d) INTEGER

4. You want to add a new row to the `users` table. Which statement is correct?

- a) `ADD INTO users (name) VALUES ('Ana');`
- b) `INSERT users SET name = 'Ana';`
- c) `INSERT INTO users (name) VALUES ('Ana');` ✓
- d) `PUT INTO users (name) VALUES ('Ana');`

5. Why is `SELECT *` considered a bad practice in production code?

- a) It returns too many rows
- b) It is not valid SQL syntax
- c) It returns columns you may not need and breaks silently when the schema changes ✓
- d) It is slower in all database engines

6. What is a soft-delete?

- a) Deleting a row and backing it up in a separate archive table
- b) Marking a row as deleted using a column instead of physically removing it ✓
- c) Deleting only some columns of a row
- d) Using `DELETE` with a `LIMIT` clause to remove rows one at a time

7. You run: `UPDATE products SET price = 0;` with no `WHERE` clause. What happens?

- a) Nothing — PostgreSQL requires a `WHERE` clause for `UPDATE`
- b) Only the first row is updated
- c) The price of every row in the table is set to 0 ✓
- d) PostgreSQL throws a syntax error

8. Which operator would you use to find all users whose last name contains "garcia", regardless of case?

- a) `LIKE '%garcia%'`
- b) `ILIKE '%garcia%'` ✓
- c) `= '%garcia%'`
- d) `CONTAINS 'garcia'`

9. What does `WHERE price BETWEEN 10 AND 50` return?

- a) Rows where price is greater than 10 and less than 50 (exclusive)
- b) Rows where price is greater than or equal to 10 and less than or equal to 50 ✓
- c) Rows where price equals exactly 10 or 50
- d) Rows where price is outside the range 10 to 50

10. You want the 3 most recently created products. Which query is correct?

- a) `SELECT * FROM products LIMIT 3;`

- b) `SELECT * FROM products ORDER BY created_at DESC LIMIT 3;` ✓
- c) `SELECT TOP 3 * FROM products ORDER BY created_at DESC;`
- d) `SELECT * FROM products ORDER BY created_at LIMIT 3;`

11. What is the difference between `COUNT(*)` and `COUNT(column)`?

- a) They are identical
- b) `COUNT(column)` counts only distinct values
- c) `COUNT(column)` counts only non-NULL values in that column; `COUNT(*)` counts all rows ✓
- d) `COUNT(column)` is always faster than `COUNT(*)`

12. You want to return only categories that have more than 10 orders. Which clause filters the groups?

- a) WHERE
- b) FILTER
- c) HAVING ✓
- d) GROUP FILTER

13. What does `WHERE email = NULL` return?

- a) All rows where email is NULL
- b) Zero rows — NULL comparisons with `=` always evaluate to UNKNOWN, not TRUE ✓
- c) A syntax error
- d) All rows where email is not NULL

14. Which is the correct way to find all users who have never logged in (`login_at` is NULL)?

- a) `WHERE login_at = NULL`
- b) `WHERE login_at == NULL`
- c) `WHERE login_at IS NULL` ✓
- d) `WHERE login_at = 'NULL'`

15. You run `SELECT AVG(score) FROM students`. Three students have a NULL score. What happens?

- a) The query fails with an error
- b) The AVG includes the NULL rows counted as 0
- c) The AVG is calculated using only the rows with a non-NULL score ✓
- d) The result is NULL because the data is incomplete

Evaluation Criteria:

- 14–15 correct: Excellent — strong command of SQL fundamentals
- 11–13 correct: Good — ready to continue; review any missed topics before the next course
- 8–10 correct: Fair — revisit filtering, aggregation, and NULL handling before proceeding
- Below 8: Needs review — re-read the course before moving on

Score Interpretation: Each question maps directly to a lesson in this course. Use wrong answers to identify exactly which section to revisit.

Section 06: Conclusion

06.0 What's Next

Content Outline:

- Course recap: what students accomplished
 - Understood the relational model: tables, rows, columns, primary keys, auto-increment, data types
 - Wrote CRUD queries: SELECT with specific columns, INSERT with RETURNING, UPDATE, DELETE
 - Filtered and sorted results with WHERE (all operator types), ORDER BY, LIMIT, OFFSET
 - Summarized data with GROUP BY, COUNT, SUM, AVG, MIN, MAX, DISTINCT, and HAVING
 - Mastered NULL: IS NULL, IS NOT NULL, NULL in comparisons, NULL in aggregations, COALESCE
- Connection to bigger picture: you can now query a PostgreSQL database confidently — reading, writing, filtering, and summarizing any single table
- Next steps: the next course introduces multi-table SQL — joining related tables, foreign keys, and entity-relationship diagrams
- Resources for further exploration:
 - PostgreSQL documentation: <https://www.postgresql.org/docs/>
 - SQLZoo interactive exercises: <https://sqlzoo.net/>
 - pgExercises: <https://pgexercises.com/>
- Encouragement: SQL is one of the most durable skills in software development — frameworks come and go, but SQL has been the language of relational databases since the 1970s and remains as relevant as ever.

Note: This is conclusion only — no theory, exercises, or assessment.